EX08 - Vertical Motion - while and Euler

Beginner Step-by-Step Guide

This guide follows the real file `ex08_vertical_motion.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Simulate vertical launch by integrating dh/dt=v and dv/dt=-g with Euler until height becomes negative.

Problem variables / elements

Name	Meaning
h	height [m]
v	vertical velocity [m/s]
t	time [s]
g	gravity [m/s^2]
dt	time step [s]

Equation or logic on paper

```
dh/dt = v
dv/dt = -g
h_new = h + v*dt
v_new = v - g*dt
t_new = t + dt
```

Manual check

First step: h=0,v=50,t=0,dt=0.1 -> h_new=5.0, v_new=49.019, t_new=0.1.

2. What we already know and what is new

Already seen:

- variables
- if/elif
- loops

New in this exercise:

- while
- simulation state
- explicit Euler
- counter i
- state update
- maximum tracking
- relative error

3. New Python concepts

1. while

Syntax: `while h >= 0:`

Repeats while the condition is True. It is appropriate when the number of required iterations is not known in advance.

2. State

Syntax: `h, v, t`

The state contains the information needed to compute the next instant.

3. Euler

Syntax: `y_new = y + derivative * dt`

Euler approximates evolution over dt using the derivative at the current state.

4. Counter

Syntax: `i += 1`

i counts iterations; it is not a physical quantity.

4. Code step by step

Step 1. Initialize state

```
t = t0
h = h0
v = v0
max_h = h0
i = 0
```

Before the while loop we need an initial state and helper variables.

Step 2. Stopping condition

```
while h >= 0:
```

While the current state is at or above ground, another step is computed.

Step 3. Compute new state

```
h_new = h + v * dt
v_new = v - g * dt
t_new = t + dt
```

First compute all new variables from the old state.

Step 4. Update maximum

```
if h_new > max_h:
    max_h = h_new
```

max_h is replaced only when a higher altitude appears.

Step 5. Advance state

```
h = h_new
v = v_new
t = t_new
i += 1
```

State k+1 becomes the current state for the next iteration.

Step 6. Validate analytically

```
analytical_h_max = h0 + v0**2 / (2 * g)
analytical_t_f = ...
analytical_v_f = ...
```

The analytical solution is independent of Euler and allows us to measure numerical error.

5. Execution flow

```
initial state
 -> while h>=0
 -> Euler step
 -> update max
 -> replace state
 -> repeat
 -> analytical comparison
```

6. Expected result / check

With dt=0.1, the source gives max_h~129.92 m, t_f~10.30 s and v_f~-51.04 m/s after 103 iterations.

7. Common beginner mistakes

- Overwriting h before all new values are computed.
- Forgetting to advance t.
- Having no stopping condition.
- Confusing numerical error with runtime errors.

8. If you remember only five things

- while is used when stopping depends on state.
- Euler turns derivatives into steps.
- The new state is computed from the current one.
- dt affects error.
- Validate against a known solution whenever possible.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.